

BÀI 30: KIỂM THỬ VÀ GỠ LỖI CHƯƠNG TRÌNH SAU BÀI NÀY EM SẼ

- Biết được một vài phương pháp đơn giản để kiểm thử chương trình.
- Biết được một vài cách gỡ lỗi đơn giản cho một chương trình.

KHỞI ĐỘNG

Bài học trước em đã biết về lỗi ngoại lệ. Tuy nhiên, một chương trình chạy không có lỗi ngoại lệ (không bị dừng đột ngột) không có nghĩa là chương trình đó không có lỗi. Những lỗi không tường minh này (còn gọi là **bug**) càng khó phát hiện và khó sửa. Theo em, làm thế nào để **kiểm thử (test)** và **gỡ lỗi (debug)** một chương trình?

1. MỘT VÀI PHƯƠNG PHÁP KIỂM THỬ CHƯƠNG TRÌNH

Hoạt động 1: Tìm hiểu một số phương pháp kiểm thử chương trình
Có nhiều phương pháp để tìm ra lỗi (bug) và ngăn chặn chúng.

a) Quan sát mã lỗi Runtime

Nếu chương trình có lỗi Runtime (đang chạy thì bị dừng), cần quan sát các mã lỗi ngoại lệ để kiểm tra vị trí dòng lệnh sinh ra lỗi, từ đó phân tích và tìm cách sửa.

b) Kiểm thử với các bộ dữ liệu test

Chương trình cần được thử với một số bộ dữ liệu đầu vào (test case) mà ta đã biết trước kết quả đầu ra. Cần chú ý:

- **Nhiều bộ test:** Cần có nhiều bộ dữ liệu test với các tiêu chí khác nhau (độ lớn, tính đa dạng).
- **Test ngẫu nhiên:** Sinh dữ liệu ngẫu nhiên trong miền xác định để tăng khả năng tìm ra lỗi.
- **Test dữ liệu ở vùng biên:** Cần kiểm tra chương trình với các giá trị ở giới hạn của miền dữ liệu (ví dụ: số lớn nhất, số nhỏ nhất, giá trị rỗng). Lỗi thường phát sinh ở các vùng biên này.

c) In các thông số trung gian

Bổ sung các lệnh print() vào giữa chương trình để in ra giá trị của các biến trung gian. Qua đó, ta có thể theo dõi luồng thực thi và kiểm tra xem thuật toán có chạy đúng như mong muốn không.

d) Sử dụng công cụ Breakpoint (Điểm dừng)

Các môi trường lập trình thường có công cụ cho phép tạo ra các "điểm dừng" (breakpoint) bên trong chương trình. Khi chạy, chương trình sẽ tạm dừng tại các điểm này, cho phép người lập trình quan sát giá trị của các biến và các thông tin khác tại thời điểm đó.

GHI NHỚ

Một số phương pháp kiểm thử và gỡ lỗi chương trình:

- Quan sát các mã lỗi của chương trình.
- Sử dụng các bộ dữ liệu test, đặc biệt là dữ liệu ở vùng biên.
- Sử dụng lệnh print() để in các biến trung gian.
- Sử dụng công cụ điểm dừng (breakpoint) trong phần mềm lập trình.

2. VÍ DỤ MINH HOẠ

Xét bài toán: Nhập hai số tự nhiên m, n và tính Ước chung lớn nhất (ƯCLN) của chúng.

Thuật toán dựa trên các quy tắc sau:

1. Nếu $m = n$, thì $ƯCLN(m, n) = m$.
2. Nếu $m > n$, thì $ƯCLN(m, n) = ƯCLN(m - n, n)$.
3. Nếu $m < n$, thì $ƯCLN(m, n) = ƯCLN(m, n - m)$.

Chương trình có thể được viết như sau:

Python

Tính ƯCLN của m, n

m = int(input("Nhập số tự nhiên m: "))

n = int(input("Nhập số tự nhiên n: "))

while m != n:

 if m < n:

 n = n - m

 else:

 m = m - n

print("Đáp số:", m)

Chúng ta sẽ tiến hành kiểm thử chương trình này.

Cách 1: In ra các giá trị trung gian

Bổ sung biến đếm k và lệnh print() để theo dõi các giá trị m, n qua mỗi vòng lặp.

Python

Tính ƯCLN của m, n

```

m = int(input("Nhập số tự nhiên m: "))
n = int(input("Nhập số tự nhiên n: "))
k = 0
while m != n:
    k = k + 1
    print("Vòng lặp", k, ": m =", m, ", n =", n)
    if m < n:
        n = n - m
    else:
        m = m - n
print("Kết thúc vòng lặp: m =", m, ", n =", n)
print("Đáp số:", m)

```

Kết quả khi nhập $m=20$, $n=16$:

Nhập số tự nhiên m: 20

Nhập số tự nhiên n: 16

Vòng lặp 1 : m = 20 , n = 16

Vòng lặp 2 : m = 4 , n = 16

Vòng lặp 3 : m = 4 , n = 12

Vòng lặp 4 : m = 4 , n = 8

Kết thúc vòng lặp: m = 4 , n = 4

Đáp số: 4

Việc quan sát sự thay đổi của các biến giúp ta hiểu rõ thuật toán và phát hiện lỗi (nếu có).

Cách 2: Sử dụng công cụ tạo điểm dừng (Breakpoint)

Ta có thể đặt một điểm dừng (breakpoint) ngay tại dòng lệnh `while m != n`:. Khi chạy chương trình với chế độ gỡ lỗi (debug), chương trình sẽ dừng lại trước mỗi vòng lặp. Tại mỗi điểm dừng, ta có thể quan sát giá trị của m và n, sau đó ghi lại vào một bảng để theo dõi.

Ví dụ với $m=20$, $n=16$:

Vòng lặp	m	n
Trước lặp	20	16

Vòng lặp	m	n
1	4	16
2	4	12
3	4	8
4	4	4
Kết thúc lặp	4	4

Kết quả cuối cùng là $m = 4$.

LUYỆN TẬP

1. Chương trình của em khi chạy phát sinh lỗi ngoại lệ `ZeroDivisionError`. Đó là lỗi gì và em sẽ xử lý lỗi này như thế nào?²
2. Chương trình sau có lỗi không³? Nếu có thì tìm và sửa lỗi.

Python

```
m = input("Nhập số tự nhiên m: ")
n = input("Nhập số tự nhiên n: ")
print("Tổng hai số đã nhập là:", m + n)
```

VẬN DỤNG

1. Chương trình sau có chức năng sắp xếp một dãy số cho trước. Hãy kiểm tra xem chương trình có lỗi không. Nếu có thì tìm và sửa lỗi.

Python

```
A = [10, 1, 5, 2, 8, 0, 4]
for i in range(len(A) - 1):
    j = i
    while j > 1 and A[j] < A[j-1]:
        A[j], A[j-1] = A[j-1], A[j]
        j = j - 1
print(A)
```

2. Để kiểm thử một chương trình, nếu chỉ bằng việc kiểm tra thông qua các bộ dữ liệu test thì có bảo đảm tìm ra hết lỗi của chương trình hay không? Vì sao?